Women Empowerment Platform — Project Explained

Oct 23, 2026 · @omar

1. The idea in one minute

This project is the **backend** (server + database) of a mobile app that connects women who work from home with customers who want homemade food, handmade products or household services.

The problem: skilled women sell through Facebook groups, WhatsApp and word of mouth. That is hard to search, has no order tracking, and customers can't tell who to trust.

Our solution: one platform where providers get verified, list what they sell, receive orders, get paid and get rated, and customers can search, order, pay, review and chat.

Who	What they can do
Guest (not logged in)	Browse categories, providers, food, crafts and services
Customer	Save addresses, place orders, pay, review completed orders, chat with providers
Provider	Upload verification documents; once accepted, sell food, crafts and services, manage slots and orders
Admin	Accept or reject providers, manage categories

What to say first in the evaluation: *"Our backend is a REST API built with Node.js, Express and MySQL (XAMPP). A mobile app sends it HTTP requests and gets JSON back."*

2. Tech stack

We kept the stack small: 4 runtime libraries, no ORM, plain SQL, and XAMPP for the database.

Tool	What it does here	Why we chose it
Node.js	Runs our JavaScript on the server	Same language as the web, huge ecosystem

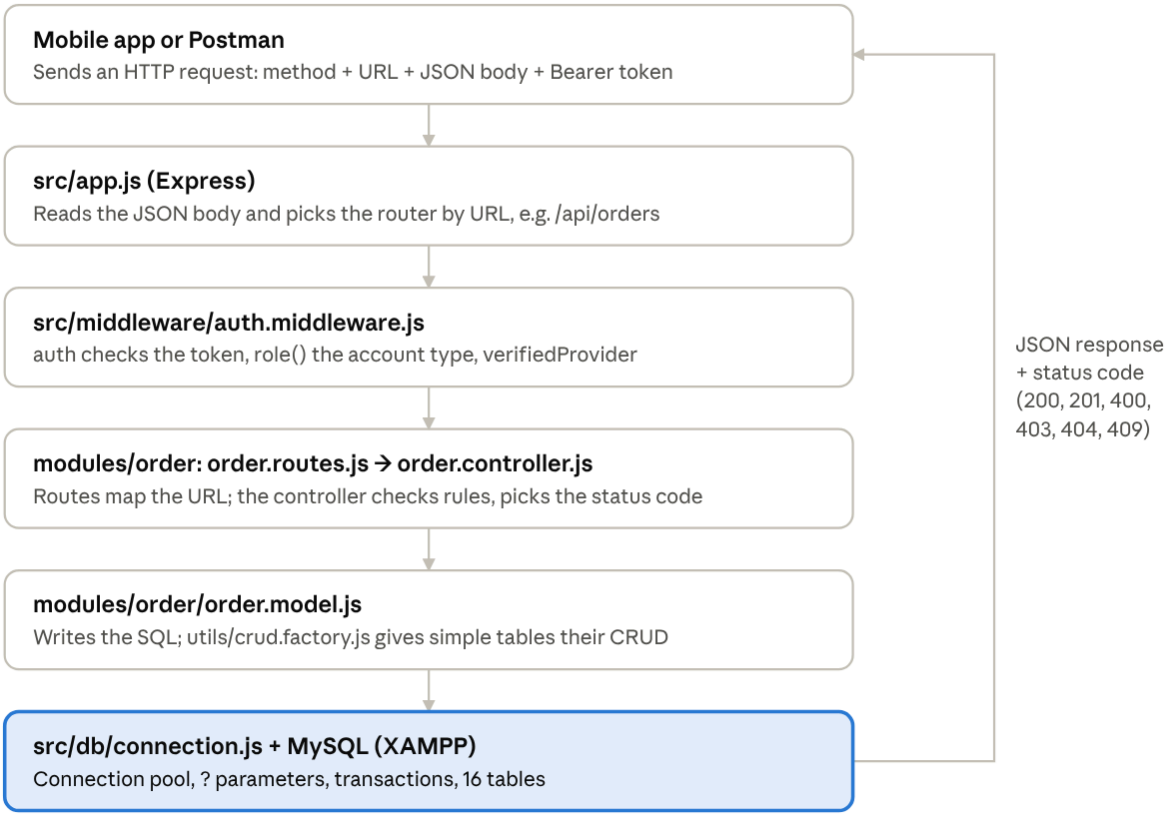
Tool	What it does here	Why we chose it
Express 5	Maps URLs to our functions, reads JSON bodies	Simple, the standard for Node APIs; v5 catches async errors for us
MySQL (XAMPP)	Stores all data in 16 related tables	Relational data (orders → items → providers) needs foreign keys and transactions; XAMPP installs it in one click
phpMyAdmin	Web page to see and import the database	Students import `database/women_platform.sql` and browse the tables without writing SQL
mysql2	The driver that sends SQL to MySQL	Lets us write real SQL with safe `?` parameters
bcryptjs	Hashes passwords	A stolen database doesn't reveal passwords
jsonwebtoken (JWT)	Login token sent with every request	No server sessions to store; works well for mobile apps
Postman	Sends requests to the API by hand or as a test run	Shows every endpoint working in the evaluation

Why no ORM (Sequelize/Prisma)? Our models write SQL directly. It is less magic, easier to explain line by line, and the database itself enforces the rules.

3. How a request travels through the code

Every request goes through the same layers, each with one job. Example: a customer places an order with `POST /api/orders`.

Every request passes the same layers down to the database and back



request flow · 6 layers

If anything goes wrong on the way down (bad input, a database rule broken), the error jumps to `src/middleware/error.middleware.js`, which turns it into the right status code and a clear message.

The code is organised **by feature**: each folder in `src/modules` holds that feature's three files — `*.routes.js` (which URLs exist and which checks run), `*.controller.js` (reads the request, applies the rules, sends the response) and `*.model.js` (the SQL).

```
index.js                    starts the server (creates the tables first)
database/
  women_platform.sql        phpMyAdmin import file: tables + demo data
src/
  app.js                    Express setup, mounts every module's routes
  db/                       connection.js, schema.sql (16 tables), seed.sql +
seed.js (demo data)
  middleware/               auth.middleware.js, error.middleware.js, not-
found.middleware.js
  utils/crud.factory.js     shared create/read/update/delete for simple tables
  modules/
    auth/  user/  address/  category/  provider/  document/
    item/  slot/  order/  payment/  review/  chat/
      e.g. order/ = order.routes.js, order.controller.js, order.model.js,
order-item.model.js
test/api.test.js            end-to-end test (npm test)
postman_collection.json
```

4. The database (16 tables from our ERD)

Every entity in the ERD is one MySQL table in `src/db/schema.sql`. Students create it by importing `database/women_platform.sql` in phpMyAdmin (tables + demo data); the server also runs the schema on start, so missing tables create themselves.

Group	Tables	Key relationships
People	`users`, `customers`, `service_providers`, `user_addresses`	Every account is a `users` row; a customer or provider row links to it 1-to-1 by `user_id`
Trust	`verification_documents`	Many documents per provider; status lives on `service_providers.verification_status`
Catalogue	`categories`, `food_items`, `craft_items`, `domestic_services`, `appointment_slots`	Each item belongs to one provider and one category; categories can have a parent; slots belong to a service
Orders	`orders`, `order_items`, `payment_transactions`	One order = one customer + one provider; many lines; many payment attempts
Feedback	`reviews`	One review per completed order (`UNIQUE order_id`)

Group	Tables	Key relationships
Chat	`chat_conversations`, `chat_messages`	One conversation per customer–provider pair; many messages

Things worth pointing at in the evaluation:

- **Primary keys** are `INT AUTO_INCREMENT`, named like the ERD: `user_id`, `order_id`…
- **Foreign keys** (`FOREIGN KEY … REFERENCES`) stop orphan data: you can't order an item that doesn't exist.
- **ENUM columns** hold fixed lists (user type, order status, payment method); phpMyAdmin shows them as dropdowns.
- **CHECK constraints** reject bad values in the database itself: `price >= 0`, `rating BETWEEN 1 AND 5`, `end_time > start_time`.
- **ON DELETE CASCADE** removes a user's addresses, items and orders with them; **ON DELETE SET NULL** keeps an order's history when an item is deleted.
- **Prices are copied into `order_items`** (`unit_price`), so changing a price later doesn't change old orders.

Small additions to the ERD: `users.password`, `verification_documents.file_url`, and `order_items.slot_id` (which slot a service order books).

5. Features, one by one

Feature	Module folder	Endpoints	How it works
Register & login	`auth/`	`POST /api/auth/register`, `/login`, `GET /auth/me`	Register creates a `users` row plus a `customers` or `service_providers` row in one transaction. Login checks the bcrypt hash and returns a JWT. Admins need `admin_secret`.
Provider verification	`document/`, `provider/`	`POST /api/documents`, `PATCH /api/providers/:id/verification`	New providers are `pending`. They upload document links; an admin accepts or rejects. Only `accepted` providers can create items (`verifiedProvider` middleware).

Feature	Module folder	Endpoints	How it works
Addresses	`address/`	`/api/addresses`	Full create/read/update/delete, only on your own addresses.
Categories	`category/`	`/api/categories`	Anyone reads; only admins create, edit, delete. Three are seeded on start.
Food, crafts, services	`item/`	`/api/foods`, `/api/crafts`, `/api/services`	One shared routes file and controller serve all three, each with its own model. Search with `q`, `category_id`, `provider_id`, `min_price`, `max_price`, `sort`.
Appointment slots	`slot/`	`/api/slots`	Providers add time slots to their services; booking one sets `is_booked`.
Orders	`order/`	`POST /api/orders`, `PATCH /api/orders/:id/status`	See below.
Payments	`payment/`	`/api/payments`	Customer records cash or card for the order total; the provider marks it `success` or `failed`. No real card gateway.
Reviews	`review/`	`/api/reviews`	Only for your own **completed** order, once. After every add/edit/delete the provider's average `rating` is recalculated.
Chat	`chat/`	`/api/conversations`, `/:id/messages`	A customer opens a chat with a provider; reading messages marks them read; the list shows `unread_count`.

Placing an order (the most important flow). `Order.create` in `src/modules/order/order.model.js` runs inside one database **transaction**:

1. Check every item exists, belongs to an accepted provider and is available.
2. Check all items come from the same provider.
3. For food/craft: check and reduce `stock_quantity`. For services: book the slot (fails if already booked).
4. Compute prices: line = unit price × quantity; service unit price = hourly rate × hours needed; add a 20 delivery fee if there is food or craft.
5. Insert the `orders` row and its `order_items`.

If any step fails, everything is rolled back: no half-orders, no stock lost.

Order status rules: provider moves `pending → accepted → in_progress → completed` (or cancels while pending); the customer can cancel only while `pending`. Cancelling puts the stock back and frees the slot.

6. Security and validation

Security works in layers; each one catches something different.

- **Passwords** are hashed with bcrypt (10 rounds) and never sent back: `user.model.js` only selects safe columns.
- **JWT:** after login the app sends `Authorization: Bearer <token>`. `auth.middleware.js` verifies it and puts `user_id`, `user_type`, `customer_id` / `provider_id` on `req.user`. Tokens last 7 days.
- **Roles:** `role('admin')`, `role('provider')` … block the wrong account type with **403**.
- **Ownership:** controllers load the row first (`loadOwn`) and compare its owner with the logged-in user; someone else's address, item or review answers **404** (we don't even confirm it exists).
- **Mass assignment protection:** each model lists which columns a request may set (`utils/crud.factory.js`: *editable* vs *fixed*). Sending `provider_id: 999` in a body is ignored.
- **SQL injection:** every value goes in as a `?` parameter, never pasted into the SQL string.
- **Database constraints** are the last line: CHECK, ENUM, NOT NULL, UNIQUE, FOREIGN KEY. MySQL runs in strict mode, so bad values are errors, not silently changed.
- **Public data stays safe:** `national_id` and passwords never appear in public endpoints.

One error handler, `src/middleware/error.middleware.js`, turns problems into clear HTTP codes:

Code	Meaning	Example
400	Bad input	Negative price, wrong status change, unknown category
401	Not logged in	No or expired token
403	Logged in, not allowed	Customer creating a category
404	Not found or not yours	Editing another provider's item, an id that doesn't exist
409	Duplicate	Email already registered, second review of an order

Code	Meaning	Example
500	Our bug	Logged on the server, generic message to the client

7. Testing and demo data

The backend is checked three ways.

1. **Automated test** — `npm test` runs `test/api.test.js` with XAMPP's MySQL running. It creates a throwaway database `women_platform_test`, walks the whole story (register → verify → sell → order → pay → review → chat, plus failure cases like wrong password, out of stock, double-booked slot, wrong role), then drops it. Your real data is never touched.
2. **Postman collection** — `postman_collection.json`, 64 requests in 11 folders, each with a status check. Run it with the Collection Runner; it saves tokens and ids between requests automatically. The last folder, *Cleanup*, deletes its test data.
3. **Demo data** — importing `database/women_platform.sql` in phpMyAdmin (or `npm run seed`) loads realistic sample data from `src/db/seed.sql`, wiping the tables first.

Demo accounts (password for all: `password123`):

Email	Role	What's there
admin@example.com	Admin	Verifies providers, manages categories
mona@example.com	Provider (accepted)	Mona's Kitchen: 4 dishes, rating 5.0
hoda@example.com	Provider (accepted)	Hoda Handmade: 3 craft items
salma@example.com	Provider (accepted)	Salma Home Services: 2 services, 5 slots, rating 4.0
nour@example.com	Provider (pending)	Waiting for admin approval
sara@example.com	Customer	Orders in several statuses, a review, a chat
omar@, laila@, karim@example.com	Customers	Orders, payments and chats

8. Running the demo

Setup, once per machine:

1. Install Node.js and XAMPP. In the XAMPP Control Panel, start **Apache** and **MySQL**.
2. Open `http://localhost/phpmyadmin` → **Import** → choose `database/women_platform.sql` → **Import**. This creates the `women_platform` database, its 16 tables and the demo data.
3. In the project folder run `npm install`, then copy `.env.example` to `.env`. The default `DATABASE_URL=mysql://root:@localhost:3306/women_platform` works with a fresh XAMPP (user `root`, no password).

On evaluation day (about 10 minutes):

1. Make sure XAMPP's MySQL is running. To reset the demo data, import the `.sql` file again (or `npm run seed`).
2. `npm run dev` — start the server; open `http://localhost:3000/api/health` and show `{"status":"ok"}`.
3. `npm test` — show the automated test passing.
4. In Postman, run the collection (skip *Cleanup*) and show 64 green requests.
5. Walk through one story by hand: log in as nour@example.com, try to create a food item (403, not verified) → log in as admin, accept her → she creates the item → log in as sara, order it → Nour accepts and completes it → Sara reviews it.
6. Open phpMyAdmin → `women_platform` and show the new rows in `orders`, `order_items` and `reviews`.
7. Open one module folder, e.g. `src/modules/order/`, and walk through its routes → controller → model files.

9. Questions the evaluator may ask

What is a REST API? A set of URLs the app calls with HTTP methods: GET reads, POST creates, PATCH updates, DELETE removes. Data goes back and forth as JSON.

What are routes, controllers, models and middleware? In each module folder: the routes file lists the URLs and which checks run; the controller reads the request, applies the rules and sends the response; the model talks to the database (the SQL). Middleware runs before the controller, e.g. checking the login token.

Why organise by feature (modules) instead of by layer? Everything about orders lives in `src/modules/order/`, so adding or changing a feature touches one folder. Bigger business rules (like placing an order) stay in the model so the controller stays short. We skipped a separate service layer: at this size it would only pass calls through.

What is CRUD and where is it? Create, Read, Update, Delete. `src/utils/crud.factory.js` gives every simple table these four operations, so many model files are only a few lines.

Why MySQL with XAMPP? It is free, installs with one click on any student computer, and phpMyAdmin lets you import, browse and check the data without writing SQL. Our relational data (orders, items, providers) needs foreign keys and transactions, which MySQL gives us.

What is a transaction and where do you use it? A group of SQL statements that all succeed or all fail. We use it for registration, placing an order, cancelling an order and sending a chat message.

How do you stop two customers buying the last cake at the same time? Inside the order transaction we lock the item row (`SELECT … FOR UPDATE`), check stock, then reduce it. The second order waits, then sees 0 left and fails.

Why hash passwords instead of encrypting them? Hashing is one-way. Even we can't read a password; at login we hash what was typed and compare.

What does the JWT contain? Can a user change it? Their ids and role, signed with `JWT_SECRET`. Any change breaks the signature, so the server rejects it.

How does the provider rating update? After each review is added, edited or deleted, one SQL statement sets `rating` to the rounded average of that provider's reviews.

What happens when an item that was ordered gets deleted? The order line keeps its price and quantity; the item id becomes NULL (`ON DELETE SET NULL`), so history survives.

What is missing / future work? Real card payments (e.g. a payment gateway), file upload for documents and images, push notifications, loyalty point rules, and the mobile app itself.